

Análisis controlado de vulnerabilidades en un servicio FTP mediante Kali Linux y virtualización

Informe de Laboratorio N.º 1

Sebastián López Osorno

Politécnico Colombiano Jaime Isaza Cadavid
Facultad de Ingenierías — Departamento de Informática
Diplomado en Ciberseguridad Informática
Docente: Edwin Andrés Ochoa Agudelo

25 de septiembre de 2026

Resumen

El presente informe documenta el desarrollo del Laboratorio N.º 1 del Diplomado en Ciberseguridad Informática, cuyo propósito fue montar un entorno de análisis controlado con Kali Linux sobre una plataforma de virtualización, desplegar de forma aislada la máquina vulnerable **“FirstHacking”** de la plataforma DockerLabs y ejecutar el ciclo completo de una evaluación de seguridad: preparación del entorno, reconocimiento, enumeración de servicios, investigación de la vulnerabilidad, validación controlada mediante una prueba de concepto y un análisis complementario de tráfico de red. Todas las actividades se realizaron sobre un objetivo autorizado (un contenedor Docker con la dirección interna 172.17.0.2), sin salir en ningún momento de la red aislada del laboratorio.

Durante el reconocimiento se identificó un único puerto expuesto, 21/tcp, ejecutando el servicio **vsftpd 2.3.4**. La investigación en fuentes reconocidas relacionó esa versión con la vulnerabilidad **CVE-2011-2523** (puerta trasera de vsftpd 2.3.4). Mediante una prueba de concepto pública se reprodujo la condición vulnerable de forma controlada, obteniendo una sesión de comandos con privilegios de **root** dentro del contenedor objetivo. Finalmente, como valor agregado, se realizó un análisis de tráfico con tcpdump que evidenció la transmisión de información **en texto plano**, demostrando de forma empírica el riesgo de emplear protocolos sin cifrado.

El informe relata además, como parte de la metodología de resolución de problemas, dos incidencias técnicas reales: la ruptura del sistema Kali Linux 2025.2 tras una actualización de paquetes —que obligó a migrar a Kali Linux 2026.2— y un error de permisos en el script de despliegue. Ambas se documentan porque forman parte del razonamiento técnico y refuerzan el valor de la virtualización como entorno recuperable y repetible.

1. Introducción y objetivos

La ciberseguridad ofensiva controlada parte de una premisa sencilla: para defender un sistema hay que comprender cómo se ataca. Kali Linux, ejecutado sobre una plataforma de virtualización, constituye el entorno idóneo para ese aprendizaje, ya que combina un conjunto maduro de herramientas de auditoría con las garantías que aporta la virtualización: **aislamiento** respecto a la máquina física, **repetibilidad** del escenario y **capacidad de recuperación** ante cualquier fallo. Gracias a ello es posible reproducir un ataque real sin poner en riesgo infraestructura productiva.

Objetivo general. Desarrollar un laboratorio controlado con Kali Linux para reconocer una máquina vulnerable, identificar los servicios expuestos, consultar vulnerabilidades conocidas y validar de manera autorizada una condición vulnerable, analizando en el proceso las capacidades de la virtualización.

Objetivos específicos.

1. Preparar y configurar de forma segura el entorno virtual de análisis.
2. Desplegar la máquina vulnerable de forma aislada e identificar su dirección interna.
3. Realizar reconocimiento y enumeración de servicios con Nmap, interpretando cada parámetro.
4. Investigar la vulnerabilidad asociada al servicio y su versión (CVE, impacto y condiciones).
5. Validar de forma controlada la explotación mediante una prueba de concepto.

6. Analizar el riesgo, proponer controles de mitigación y documentar técnicamente todo el proceso.

2. Condiciones de seguridad

La práctica se ejecutó respetando estrictamente las condiciones establecidas para un laboratorio controlado:

- Las pruebas se realizaron **únicamente** sobre la máquina vulnerable autorizada para la actividad.
- El objetivo se mantuvo **aislado** en la red interna de Docker (172.17.0.0/16), sin exposición a redes institucionales, productivas ni de terceros.
- No se emplearon direcciones IP públicas ni sistemas ajenos al laboratorio.
- No se realizó persistencia, movimiento lateral ni acceso a otros sistemas.
- La prueba se detuvo una vez obtenida evidencia suficiente del objetivo académico.

Estas condiciones no son un formalismo: delimitan el alcance ético y legal del ejercicio y garantizan que una actividad ofensiva se mantenga como una práctica de aprendizaje y no como una amenaza real.

3. Entorno virtual y preparación

El laboratorio se montó sobre **VMware Workstation Pro 17**, un hipervisor de tipo 2 que permite ejecutar varios sistemas operativos huésped de forma simultánea y aislada. El sistema atacante fue **Kali Linux**, comunicado con el objetivo a través de la red interna que gestiona Docker. El objetivo, en cambio, se ejecuta como **contenedor**, de modo que en el mismo escenario conviven las dos grandes tecnologías de aislamiento —máquina virtual y contenedor—, cuya diferencia se analiza en la sección 13.

Descripción del entorno y de la red. El escenario quedó configurado de la siguiente manera:

Componente	Detalle
Hipervisor	VMware Workstation Pro 17
Sistema atacante	Kali Linux 2026.2 (máquina virtual)
Recursos de la máquina virtual	8 GB de RAM, 8 procesadores y disco de 100 GB
Objetivo	Contenedor Docker “FirstHacking”
Segmento de red aislada	Red interna de Docker 172.17.0.0/16
Dirección IP del objetivo	172.17.0.2
Puerta de enlace / interfaz de Kali	172.17.0.1 / 192.168.5.136

La comunicación entre Kali y el objetivo se mantuvo dentro de este segmento aislado, sin exposición a la red institucional ni a terceros.

3.1 Despliegue inicial del objetivo

Se utilizó la plataforma **DockerLabs** para descargar y desplegar la máquina vulnerable “FirstHacking”. El script de despliegue construye el contenedor a partir de una imagen y le asigna automáticamente una dirección en la red interna de Docker; en este caso, la dirección **172.17.0.2**, que constituye el objetivo autorizado durante todo el ejercicio.

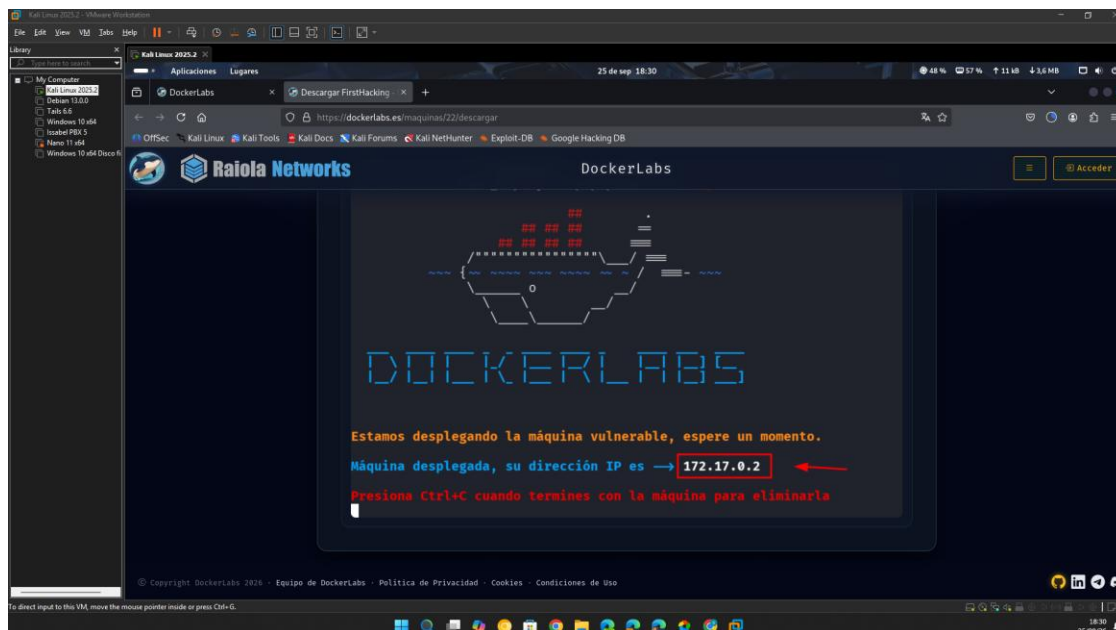


Figura 1

Figura 1. Plataforma DockerLabs con la máquina “FirstHacking” desplegada; se resalta la dirección IP interna asignada al objetivo, 172.17.0.2.

3.2 Incidencia técnica: actualización y ruptura de Kali 2025.2

Como parte del mantenimiento inicial se ejecutó una actualización completa del sistema (`apt update && apt upgrade`) sobre **Kali Linux 2025.2**. Durante el proceso se descargaron y desempaquetaron paquetes de base del sistema (entre ellos `libc6`, `locales` y componentes de `texlive`).

(snapshot) del entorno antes de iniciar las pruebas; en esta práctica la reinstalación desde la ISO cumplió esa misma función de recuperación, devolviendo el sistema a un estado limpio y operativo.

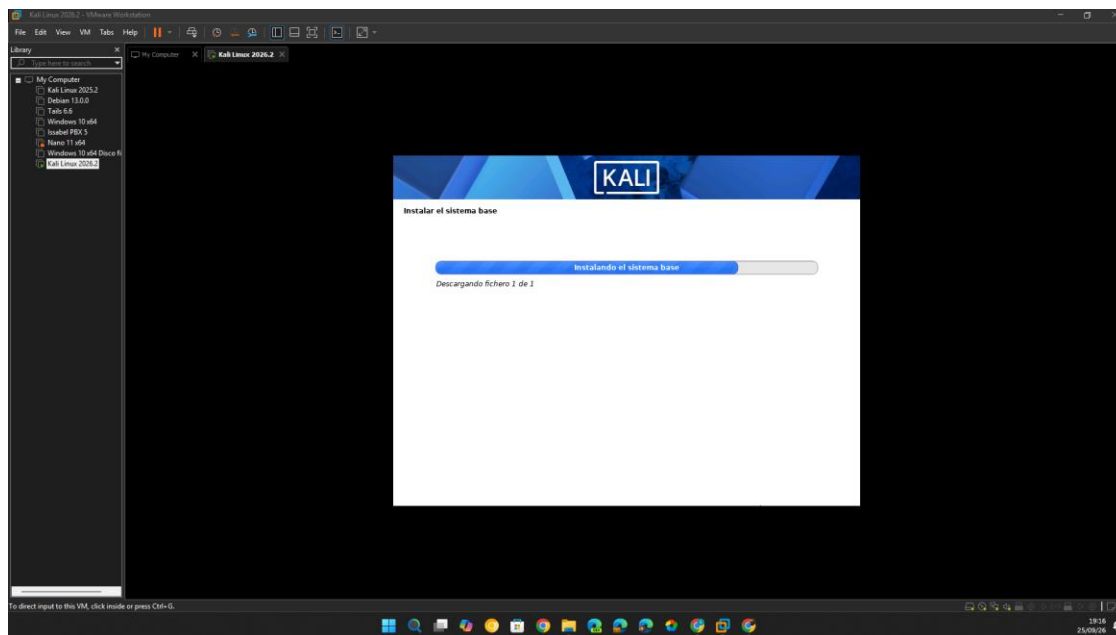


Figura 4

Figura 4. Instalación del sistema base de Kali Linux 2026.2 desde la imagen ISO.



Figura 5

Figura 5. Entorno de Kali Linux 2026.2 operativo tras la migración.

3.3 Recursos de la máquina virtual e instantánea de seguridad

Se verificaron los recursos asignados a la máquina virtual y, como buena práctica del laboratorio, se tomó una **instantánea (snapshot)** del entorno **antes de iniciar las pruebas**. La instantánea permite restaurar el sistema a un estado limpio y conocido en cualquier momento, lo que garantiza la repetibilidad del escenario y la recuperación ante cualquier alteración.

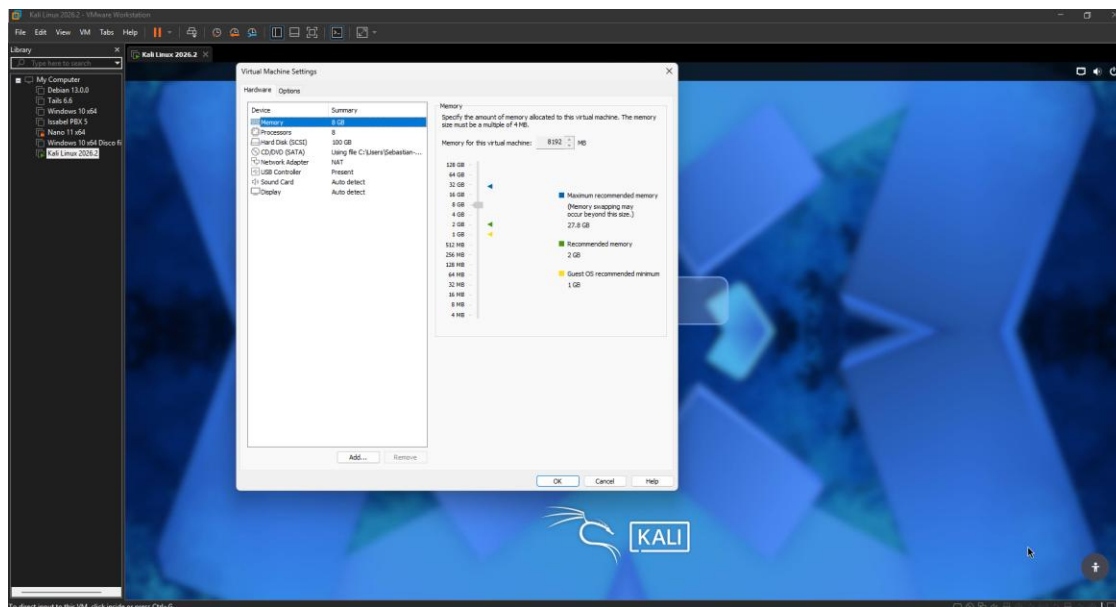


Figura 6

Figura 6. Recursos asignados a la máquina virtual: 8 GB de memoria RAM, 8 procesadores, disco de 100 GB y adaptador de red.

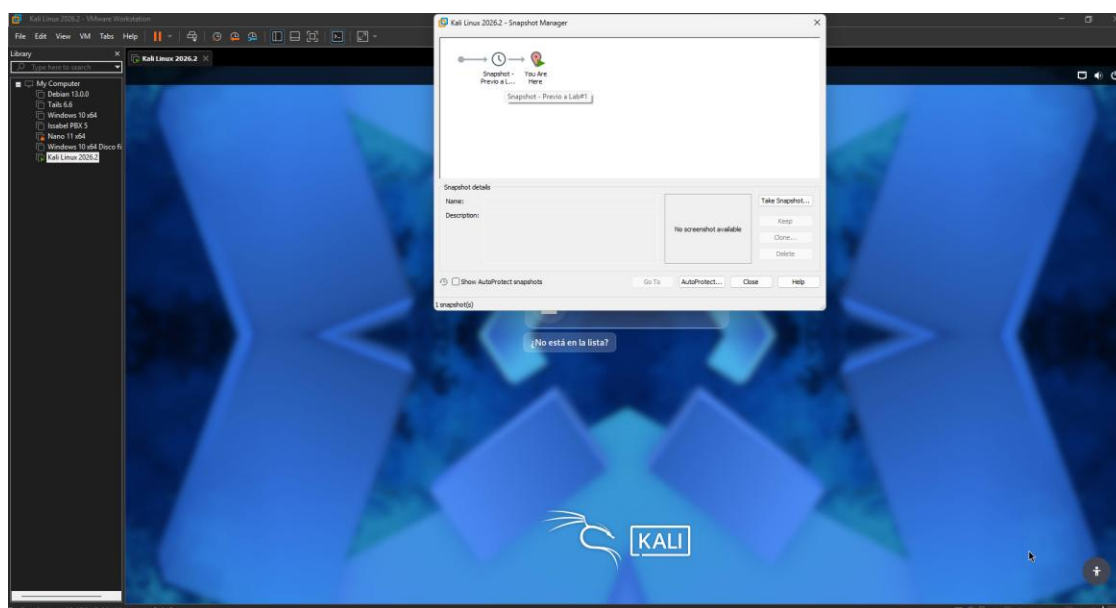


Figura 7

Figura 7. Instantánea del entorno tomada antes de las pruebas: «Snapshot – Previo a Lab#1».

4. Despliegue de la máquina vulnerable

4.1 Exploración del paquete descargado

El objetivo se distribuye como un archivo comprimido `firsthacking.zip`. Antes de ejecutar nada se inspeccionó el paquete —una buena práctica de seguridad frente a cualquier archivo descargado— verificando su naturaleza con el comando `file` y comprobando la herramienta de descompresión disponible.

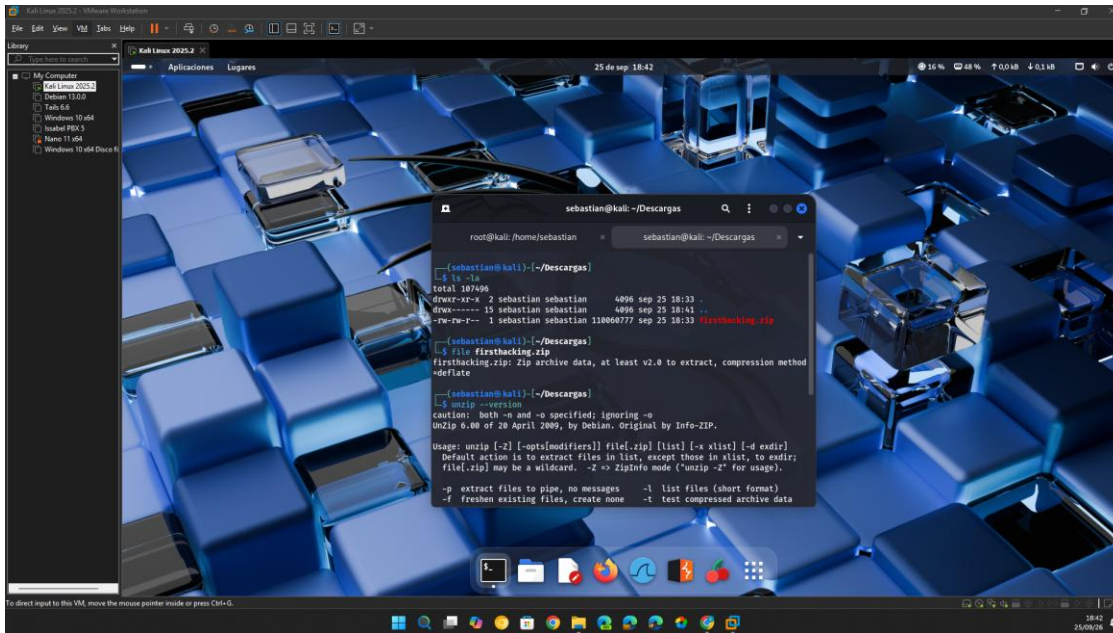


Figura 8

Figura 8. Identificación del archivo con `file` `firsthacking.zip` (Zip archive data) y verificación de `unzip`.

Al descomprimir, el paquete reveló tres elementos: el script de despliegue `auto_deploy.sh`, la imagen del contenedor `firsthacking.tar` (≈ 273 MB) y el propio `.zip`. El script es el orquestador que importa la imagen a Docker y levanta el contenedor.

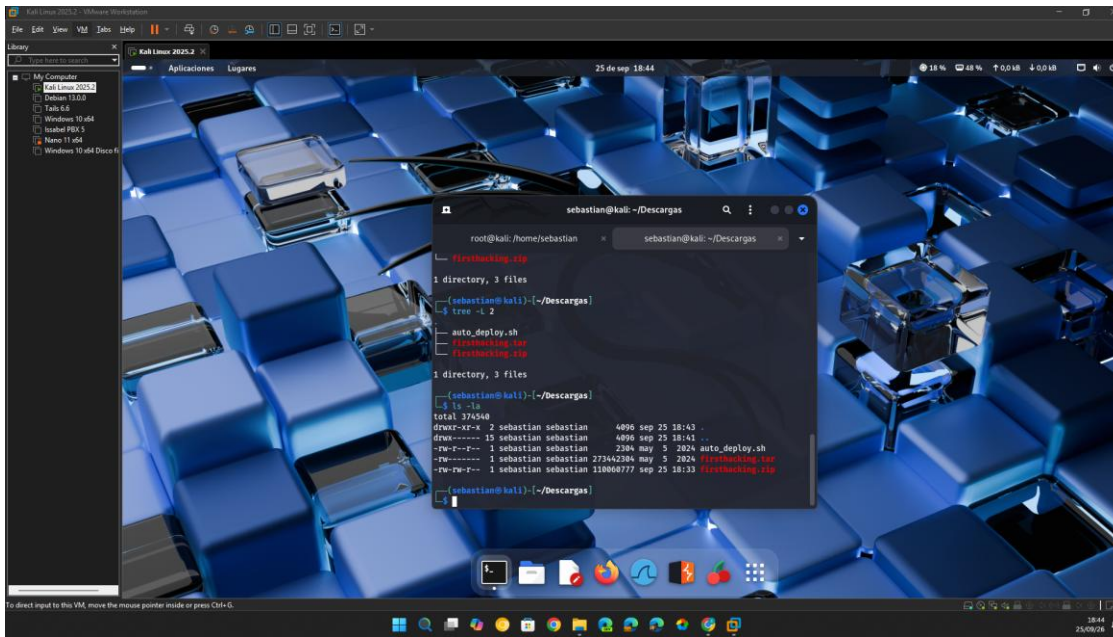


Figura 9

Figura 9. Estructura del paquete: auto_deploy.sh, firsthacking.tar y firsthacking.zip.

4.2 Troubleshooting de permisos y privilegios

El script auto_deploy.sh carecía de permiso de ejecución. Se asignó con `chmod 755` y, al ejecutarlo sin argumentos, el propio script indicó su modo de uso (Uso: ./auto_deploy.sh <archivo_tar>), lo que confirmó que esperaba recibir la imagen .tar como parámetro.

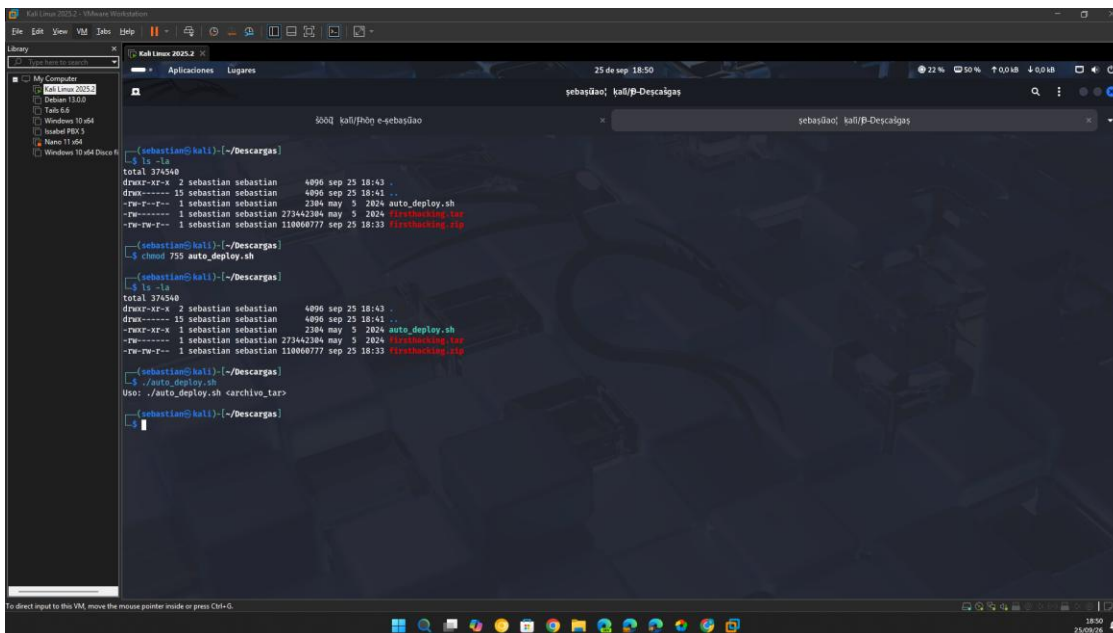


Figura 10

Figura 10. Asignación de permisos con `chmod 755 auto_deploy.sh` y mensaje de uso del script.

La documentación oficial de la plataforma confirma que el despliegue requiere **privilegios elevados**, ejecutándose como `sudo bash auto_deploy.sh <archivo>.tar`, ya que el script necesita gestionar el motor Docker (importar la imagen, crear la red y levantar el contenedor), operaciones reservadas al superusuario.

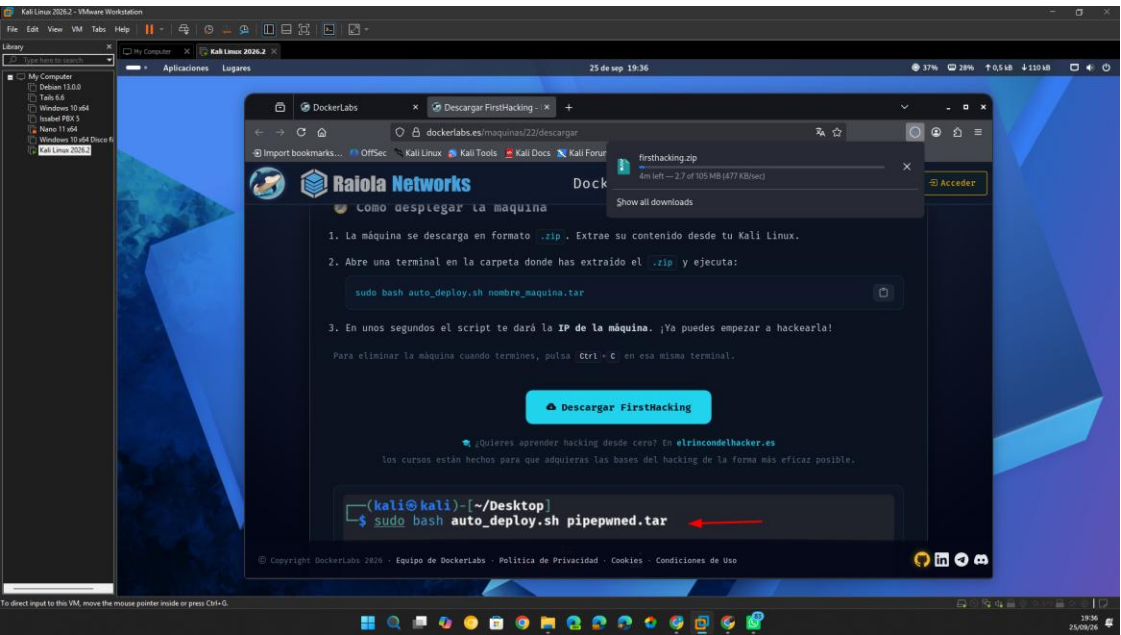


Figura 11

Figura 11. Instrucciones oficiales de despliegue, que indican el uso de sudo.

Ya sobre Kali 2026.2 se repitió el procedimiento de forma limpia: descompresión y asignación de permisos.

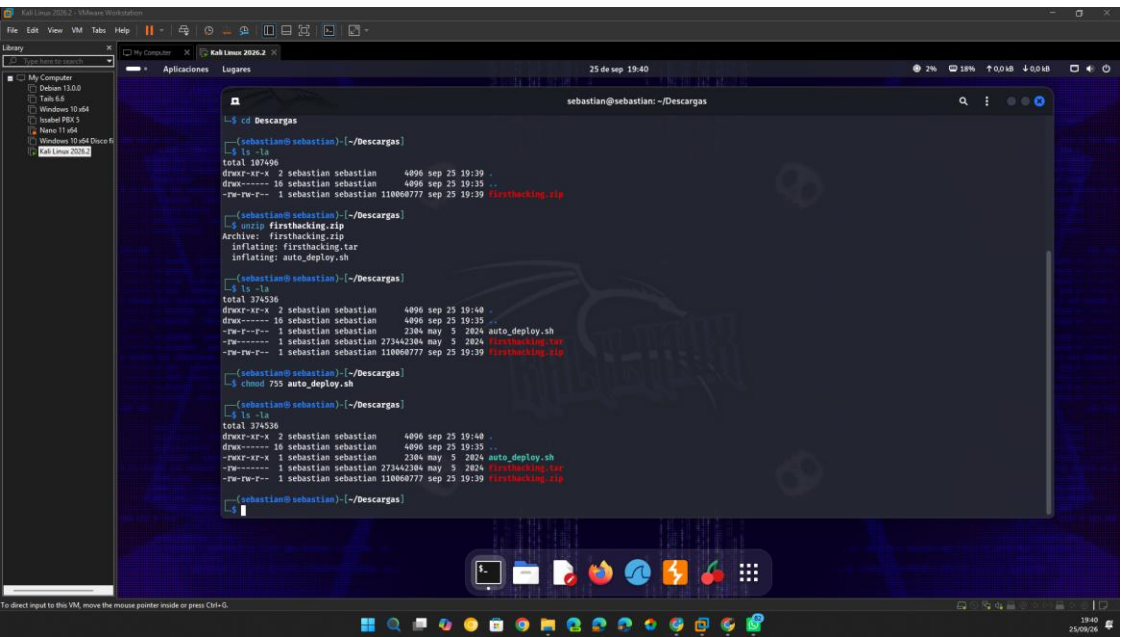


Figura 12

Figura 12. Descompresión de firsthacking.zip y asignación de permisos en Kali Linux 2026.2.

Ejecutado con privilegios, el despliegue desencadenó la **instalación de Docker** y sus dependencias (docker.io, containerd, runc, docker-cli, entre otras), tras lo cual el contenedor objetivo quedó activo en 172.17.0.2.

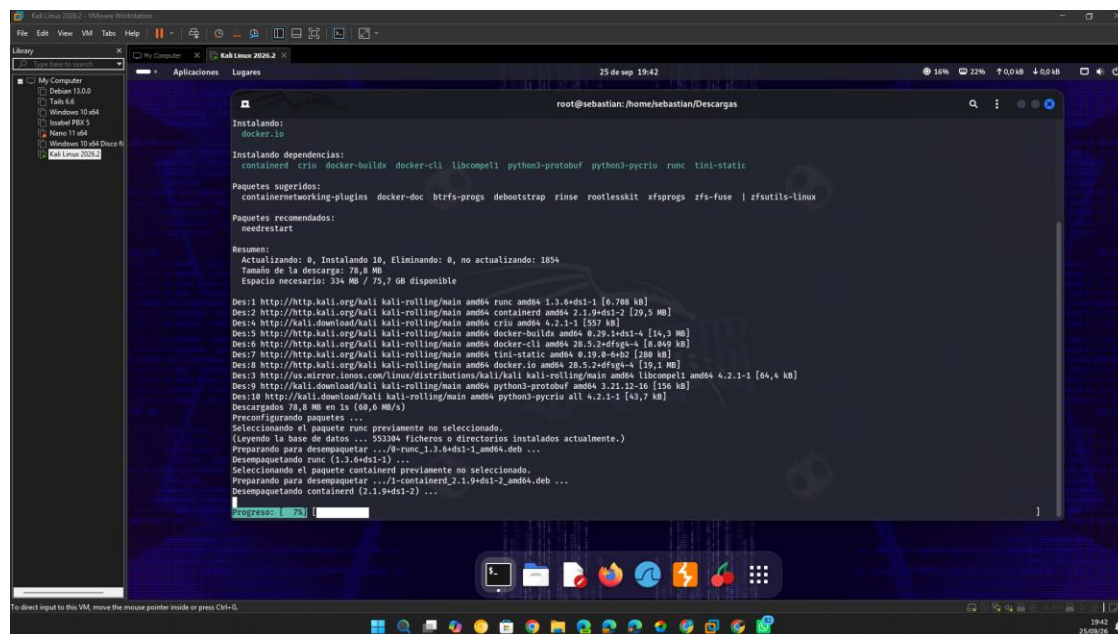


Figura 13

Figura 13. Instalación de Docker y dependencias requeridas por el script de despliegue.

5. Reconocimiento inicial

Con el objetivo activo se inició la fase de reconocimiento, cuyo fin es descubrir qué servicios expone la máquina. Esta etapa también dejó registro, como parte de la metodología de resolución de problemas, de varios **errores de sintaxis** cometidos y corregidos durante el uso de Nmap (por ejemplo nmap -Ss en lugar de -sS, o la orden inexistente map), así como del aviso Host seems down ... try -Pn, que orientó el ajuste de los parámetros del escaneo. Documentar estos tanteos refleja el proceso real de aprendizaje de la herramienta.

Comprobación de conectividad. Como el objetivo bloquea las sondas ICMP (ping), la conectividad se validó por la misma vía que Nmap emplea en la red local: la respuesta ARP del host (Host is up, received arp-response, visible en el escaneo de la Figura 16), que confirma que el objetivo está activo antes de proseguir.

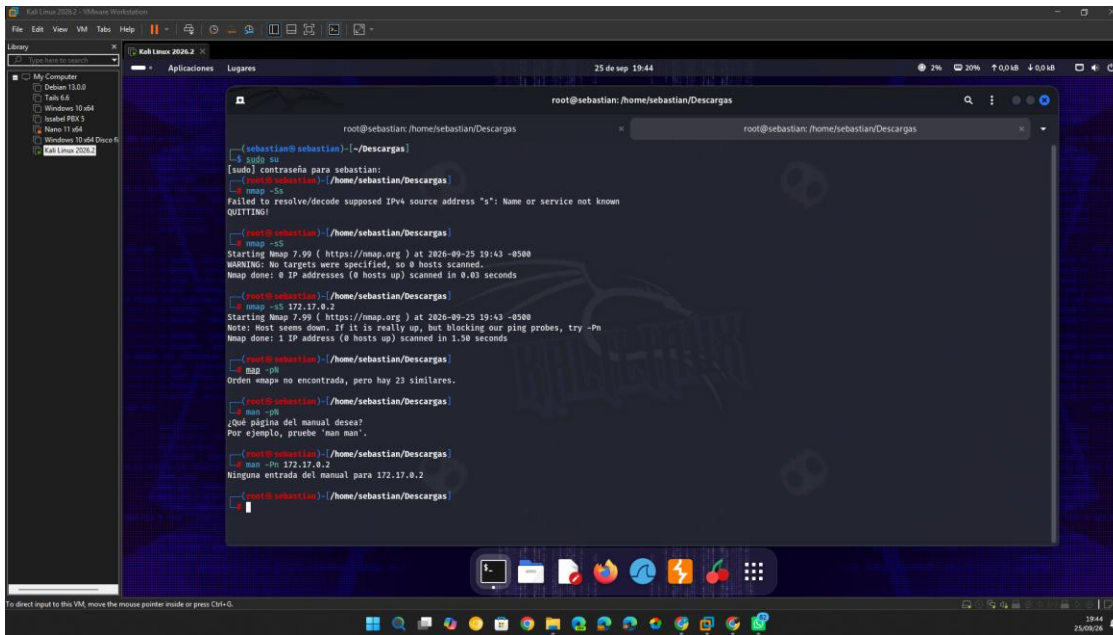


Figura 14

Figura 14. Intentos iniciales de escaneo con Nmap y corrección de errores de sintaxis; el objetivo aparece como “caído” hasta ajustar los parámetros.

6. Escaneo y enumeración con Nmap

Se ejecutó un escaneo exhaustivo de los 65 535 puertos TCP, deshabilitando el descubrimiento por ping (-Pn) —porque el objetivo bloqueaba las sondas ICMP— y guardando la salida en un fichero (-oN):

```
nmap -sS -Pn -n -vvv -p- --min-rate=5000 -oN primer_escaneo 172.17.0.2
```

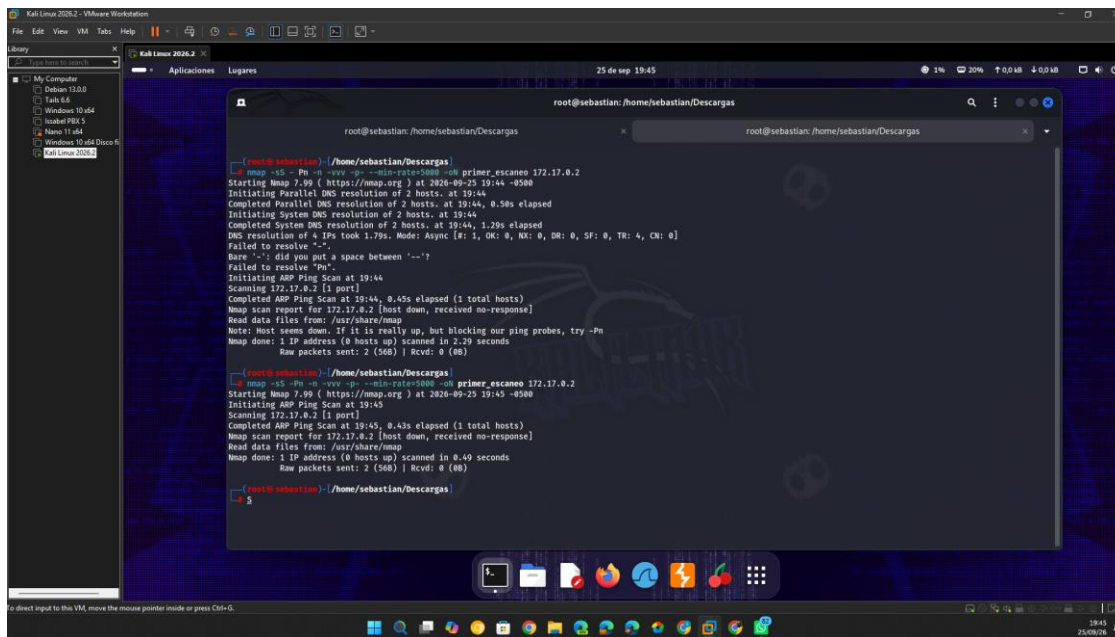


Figura 15

Figura 15. Escaneo completo de puertos con Nmap sobre 172.17.0.2.

Interpretación de los parámetros empleados:

Parámetro	Función
-sS	Escaneo TCP SYN (“half-open”): rápido y discreto; no completa el saludo de tres vías.
-Pn	Omite el descubrimiento por ping y trata al host como activo (necesario porque bloquea ICMP).
-p-	Escanea el rango completo de puertos (1–65535).
-sV	Detección de servicio y versión sobre los puertos abiertos.
-n	Evita la resolución DNS, lo que acelera el escaneo.
--min-rate=5000	Fuerza un mínimo de 5000 paquetes por segundo (rendimiento).
-oN	Guarda la salida en formato normal en el fichero indicado.

Añadiendo -sV se identificó, sobre el único puerto abierto, el servicio y su versión exacta:

```

PORT      STATE SERVICE REASON      VERSION
21/tcp    open  ftp      syn-ack ttl 64 vsftpd 2.3.4
Service Info: OS: Unix

```

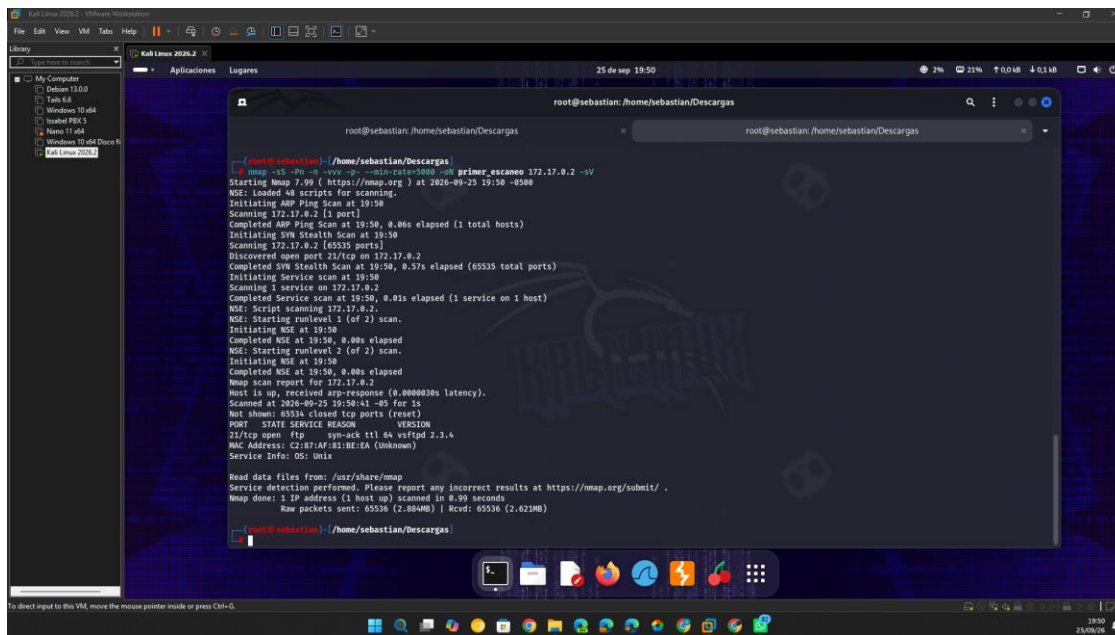


Figura 16

Figura 16. Detección de versión con -sV: puerto 21/tcp abierto ejecutando **vsftpd 2.3.4**.

Resultado del reconocimiento. De 65 535 puertos, solo 21/tcp está abierto, ejecutando el servicio FTP **vsftpd 2.3.4** sobre un sistema Unix. Un servicio FTP de una versión antigua y concreta constituye una **superficie de ataque** de manual: expone un punto de entrada, revela la versión exacta del software —lo que facilita la búsqueda de exploits dirigidos— y emplea un protocolo que transmite sin cifrado. La combinación de estos tres factores convierte a este único puerto en el foco de todo el análisis posterior.

Comparación de los escaneos. El reconocimiento siguió una progresión de menor a mayor detalle: un primer escaneo general para descubrir los puertos abiertos, un escaneo TCP SYN (-ss) dirigido para confirmar el estado del puerto y, por último, un escaneo de versión (-sV) para identificar el software exacto. La comparación es reveladora: los dos primeros indican *qué* puerto está abierto, pero solo el tercero aporta el dato decisivo —la versión **vsftpd 2.3.4**—, que es precisamente lo que habilita la búsqueda de la vulnerabilidad concreta.

7. Investigación de la vulnerabilidad

7.1 Identificación de la vulnerabilidad

La versión **vsftpd 2.3.4** es conocida por contener una **puerta trasera (backdoor)** que fue introducida maliciosamente en el código fuente del proyecto durante un compromiso de su servidor de distribución. La vulnerabilidad está catalogada como **CVE-2011-2523**. Esta correspondencia entre la versión detectada y el identificador se contrastó en la Base de Datos Nacional de Vulnerabilidades (NVD/NIST) y en MITRE, fuentes de referencia en la gestión de vulnerabilidades.

- **Descripción.** El código comprometido abre una puerta trasera: cuando el nombre de usuario enviado en el login FTP contiene la secuencia :) (una “carita sonriente”), el servicio abre un intérprete de comandos vinculado al puerto **6200/tcp**.
- **Impacto.** Ejecución remota de comandos con privilegios de **root**, es decir, el compromiso total del sistema afectado (confidencialidad, integridad y disponibilidad).
- **Condiciones.** El servicio vulnerable debe ser accesible por red; no se requiere una autenticación válida, porque el disparador es la propia cadena del nombre de usuario.
- **Corrección.** El proyecto retiró el código malicioso poco después de detectarlo; las versiones posteriores (la rama 2.3.x corregida y toda la **serie 3.x**) no contienen la puerta trasera. La mitigación efectiva, por tanto, es actualizar a una versión soportada del servidor FTP.

7.2 Búsqueda de referencias de explotación

Se localizó una prueba de concepto pública mediante Searchsploit / Exploit-DB, obteniéndose el script en Python **49757.py** (Exploit-DB, EDB-ID 49757). Antes de ejecutarlo se verificó su modo de uso y el intérprete disponible (Python 3.13.12), confirmando que el script requiere la dirección del objetivo como argumento.

```

root@sebastian: /home/sebastian/Descargas
root@sebastian: /home/sebastian/Descargas

root@sebastian: /home/sebastian/Descargas
ls -la
total 37408
drwxr-xr-x 2 sebastian sebastian 4096 sep 25 19:48
drwx----- 16 sebastian sebastian 4096 sep 25 19:41
-rwxr-xr-x 1 root root 1181 sep 25 19:48 49757.py
-rwxr-xr-x 1 sebastian sebastian 2304 may 5 2024 auto_deploy.sh
-rw----- 1 sebastian sebastian 273442384 may 5 2024 firsthooking.tar
-rw-r--r-- 1 sebastian sebastian 118000777 sep 25 19:39 firsthooking.zip
-rw-r--r-- 1 root root 700 sep 25 19:50 primer_escaneo

root@sebastian: /home/sebastian/Descargas
python3 49757.py
usage: 49757.py [-h] host
49757.py: error: the following arguments are required: host

root@sebastian: /home/sebastian/Descargas
python3 49757.py
Orden «python3» no encontrada. Quizá quiso decir:
la orden «shyle» del paquete deb «shyle»
la orden «sys» del paquete deb «python3»
la orden «sys» del paquete deb «sys»
la orden «python» del paquete deb «python3-common»
Pruebe con apt install «nombre del paquete deb»

root@sebastian: /home/sebastian/Descargas
python --version
Python 3.13.12

root@sebastian: /home/sebastian/Descargas
python3 --version
Python 3.13.12

root@sebastian: /home/sebastian/Descargas
python --version
Python 3.13.12

```

Figura 17

Figura 17. Prueba de concepto 49757.py en el directorio de trabajo y verificación del intérprete Python.

La revisión de la cabecera del script confirma su asociación con **CVE-2011-2523** y documenta su mecanismo: se conecta al FTP (puerto 21), envía un nombre de usuario que incluye el disparador :) y, acto seguido, se conecta al puerto **6200**, donde el propio servidor ha abierto un intérprete de comandos. *(Con criterio de buenas prácticas, este informe describe el mecanismo con fines académicos y muestra la evidencia, sin reproducir el código completo del exploit.)*

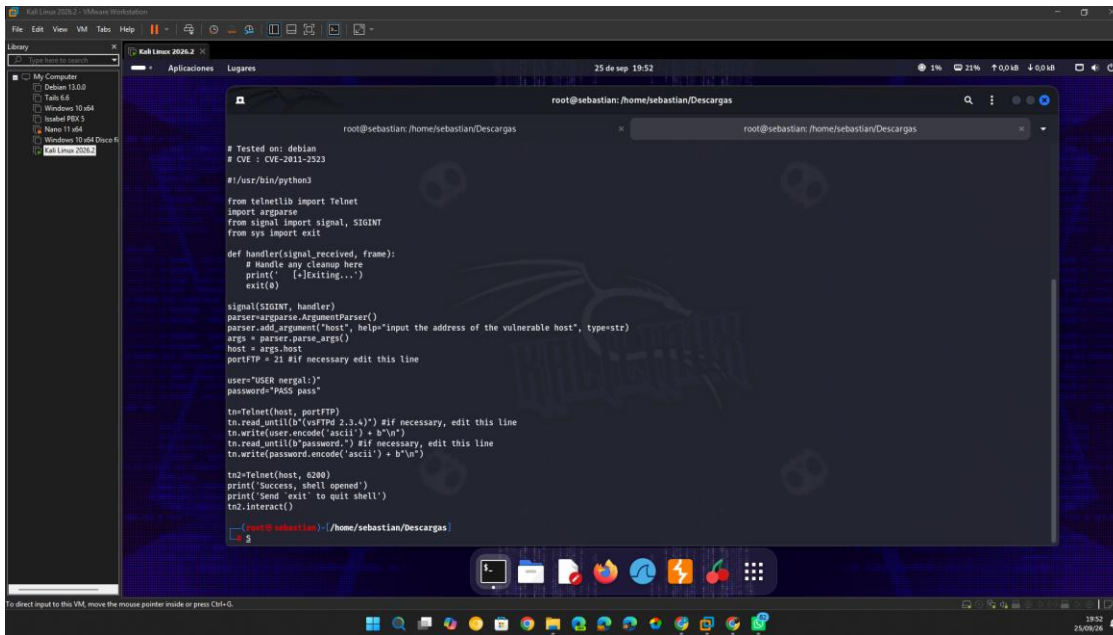


Figura 18

Figura 18. Cabecera y lógica de la prueba de concepto, con la referencia a CVE-2011-2523.

8. Validación controlada (prueba de concepto)

La explotación se ejecutó **exclusivamente** sobre la máquina vulnerable autorizada y dentro de la red aislada. El primer intento devolvió `ConnectionRefusedError: [Errno 111] Connection refused` — el canal de la puerta trasera aún no estaba disponible en ese instante—; un segundo intento tuvo éxito y devolvió la sesión de comandos con el mensaje “Success, shell opened”.

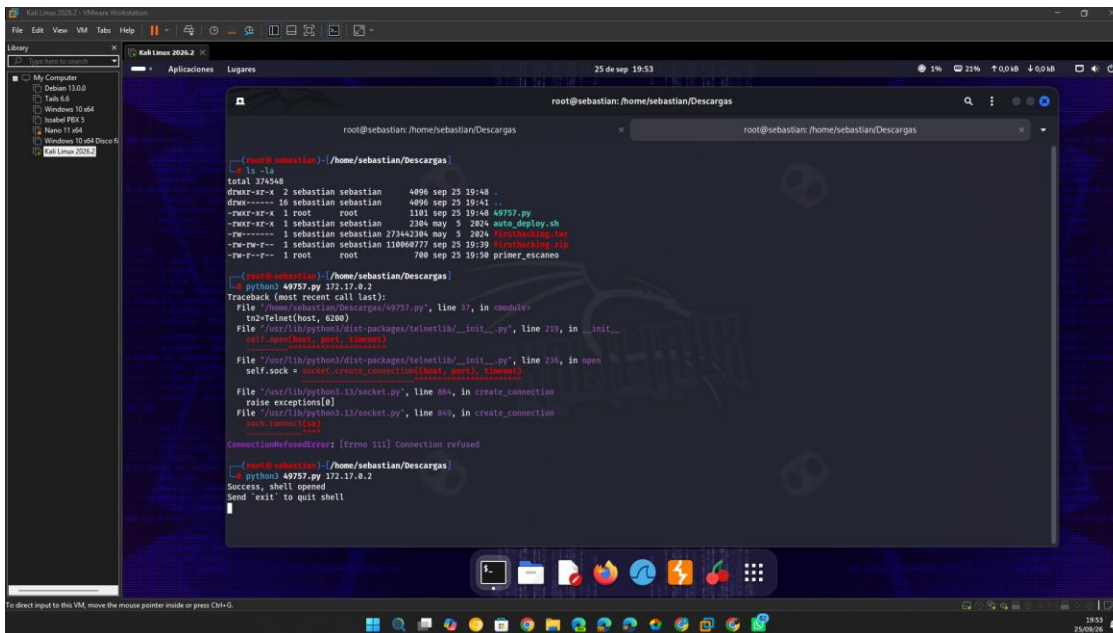


Figura 19

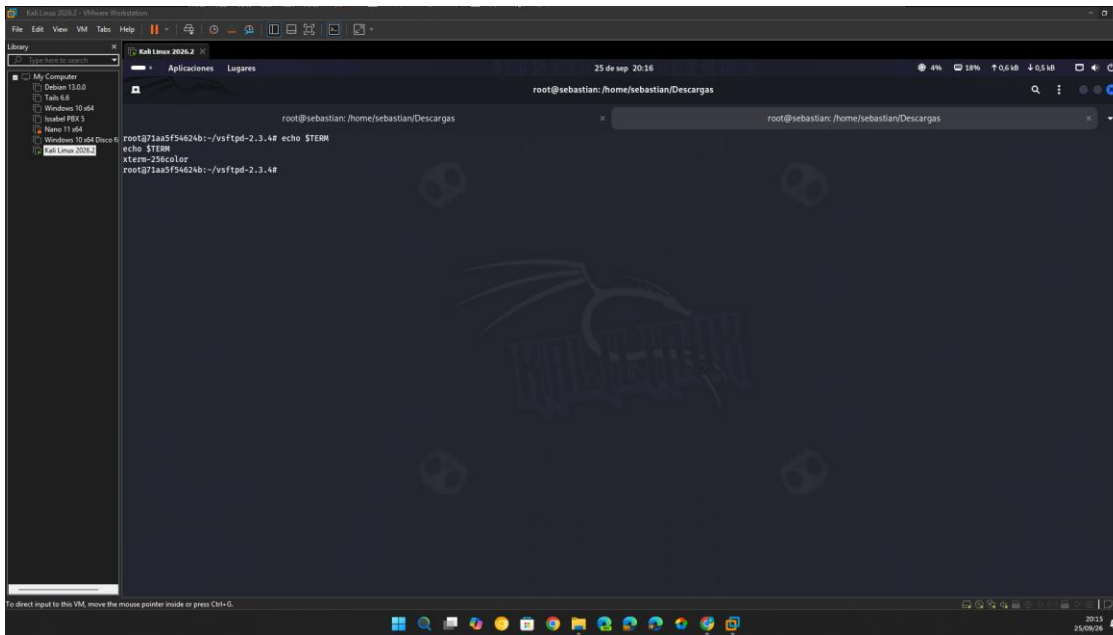


Figura 24

Figura 24. Confirmación de la variable de entorno TERM=xterm-256color.

Verificación del nivel de acceso. Cumpliendo el procedimiento del laboratorio, se comprobó el usuario de la sesión, el nombre del sistema y la información de red disponible:

- **Usuario de la sesión:** root (comando whoami).
- **Nombre del sistema (host del contenedor):** 71aa5f54624b.
- **Dirección del objetivo:** 172.17.0.2.
- **Host atacante (Kali):** puerta de enlace de Docker 172.17.0.1 e interfaz 192.168.5.136.

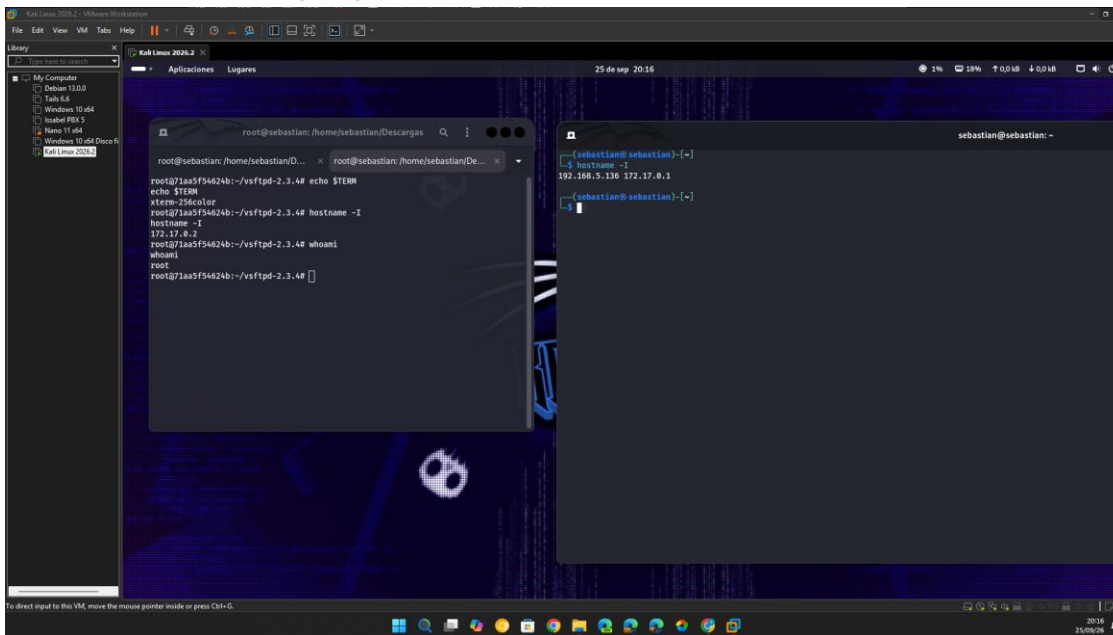
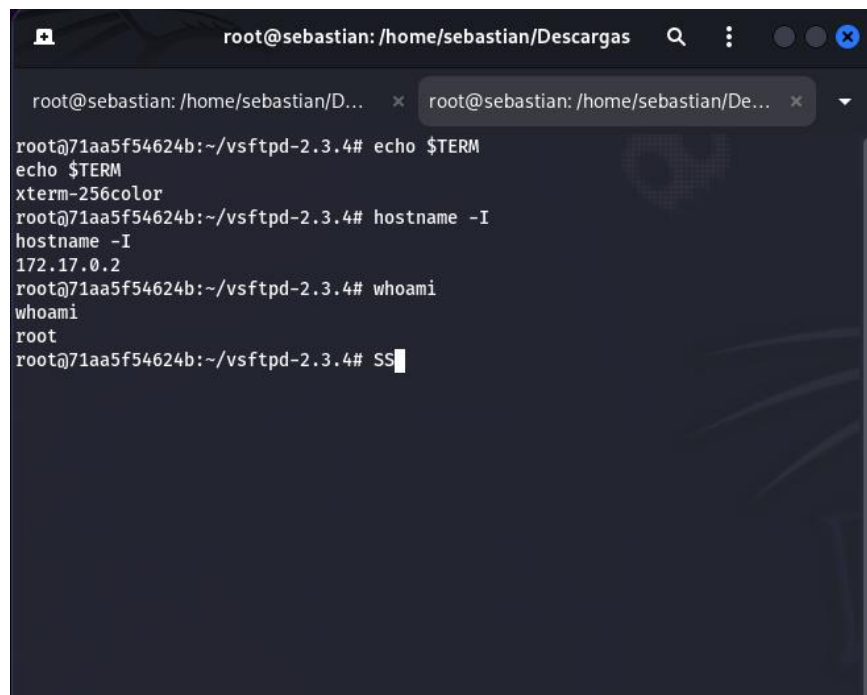


Figura 25

Figura 25. Verificación simultánea: en el objetivo whoami devuelve root y hostname -I 172.17.0.2; a la derecha, el host Kali con 172.17.0.1.



```
root@sebastian: /home/sebastian/D... x root@sebastian: /home/sebastian/De... x
root@71aa5f54624b:~/vsftpd-2.3.4# echo $TERM
echo $TERM
xterm-256color
root@71aa5f54624b:~/vsftpd-2.3.4# hostname -I
hostname -I
172.17.0.2
root@71aa5f54624b:~/vsftpd-2.3.4# whoami
whoami
root
root@71aa5f54624b:~/vsftpd-2.3.4# SS
```

Figura 26

Figura 26. Detalle de la verificación de acceso con privilegios de root sobre el objetivo.

Conforme a las condiciones de seguridad, **una vez demostrado el acceso** se dio por cumplido el objetivo académico; no se realizó persistencia, movimiento lateral ni acceso a otros sistemas.

9. Actividad complementaria: análisis de tráfico de red

Como valor agregado se analizó el tráfico intercambiado con el objetivo, con el fin de evidenciar de forma empírica el riesgo de utilizar **protocolos sin cifrado**. A diferencia de SFTP o FTPS, el protocolo FTP transmite en texto plano tanto el banner y las credenciales como —en este escenario— los datos que circulan por el canal de la puerta trasera. La captura se realizó con:

```
sudo tcpdump -i docker0 -n host 172.17.0.2 -A
```

El parámetro -A muestra el contenido de los paquetes en ASCII, e -i docker0 selecciona la interfaz del puente de Docker por la que viaja todo el tráfico hacia el contenedor. Al generar en paralelo una conexión FTP (ftp 172.17.0.2, con usuario anonymous), la captura mostró **en claro** el banner del servicio 220 (vsFTPd 2.3.4) y el resultado del intento de autenticación (530 Login incorrect).

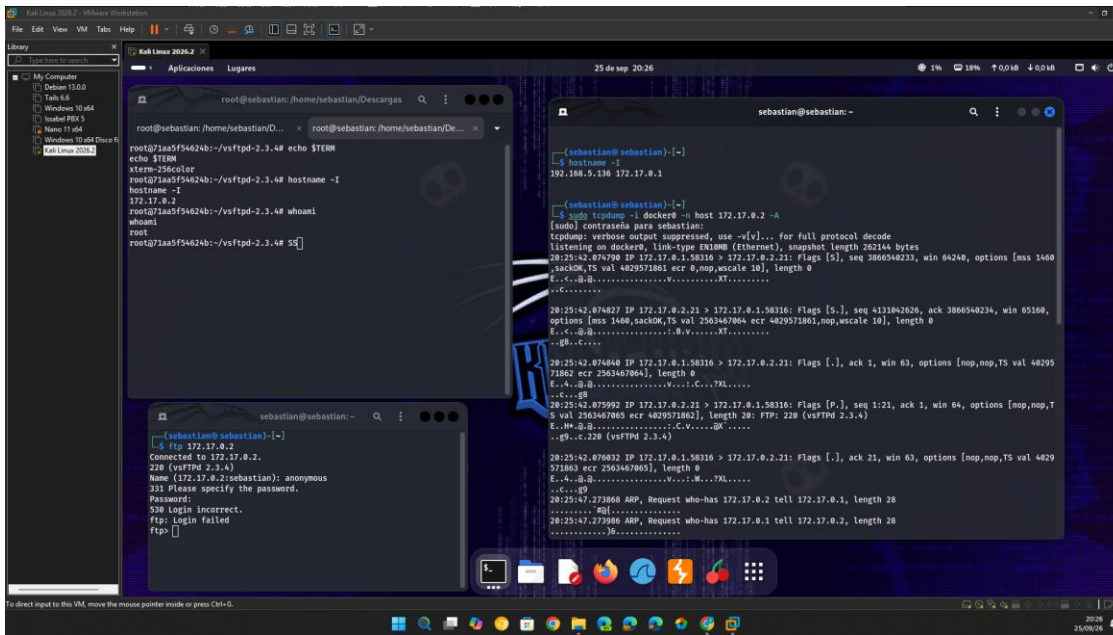


Figura 27

Figura 27. Captura con tcpdump: el banner 220 (vsFTPd 2.3.4) y el diálogo FTP viajan en texto plano.

El análisis del canal de la puerta trasera (6200/tcp) evidenció que **los comandos y su salida** también circulan sin cifrado: se observan directamente en la captura la petición de listado de directorio, su contenido y las respuestas del sistema, además del tráfico ARP asociado a la resolución de direcciones dentro de la red de Docker.

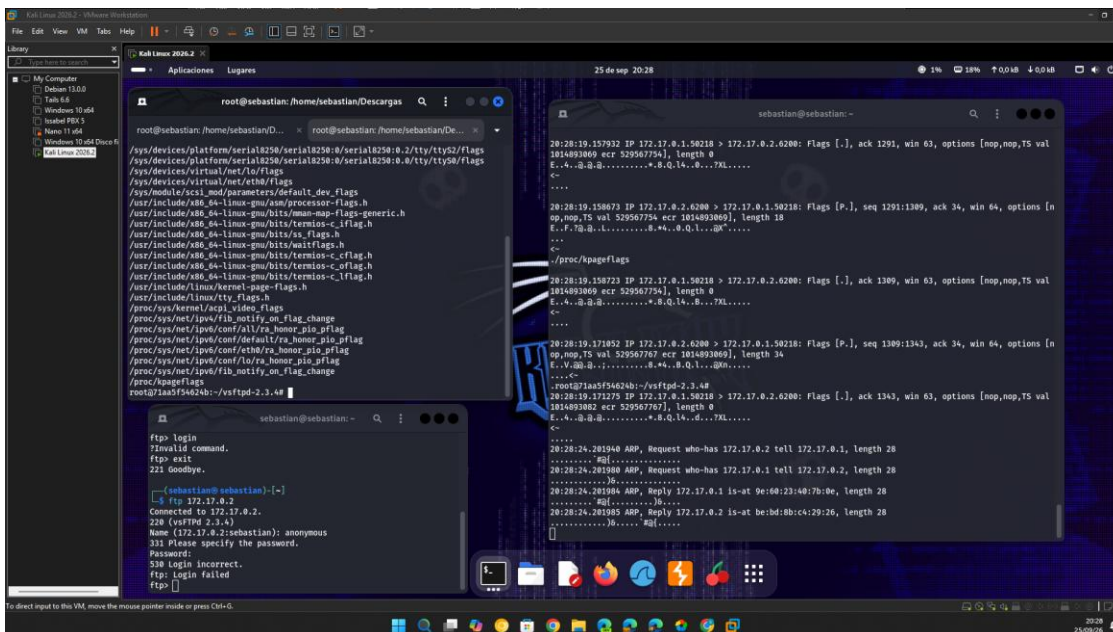


Figura 28

Figura 28. Tráfico del canal de la shell (puerto 6200) legible en texto plano.

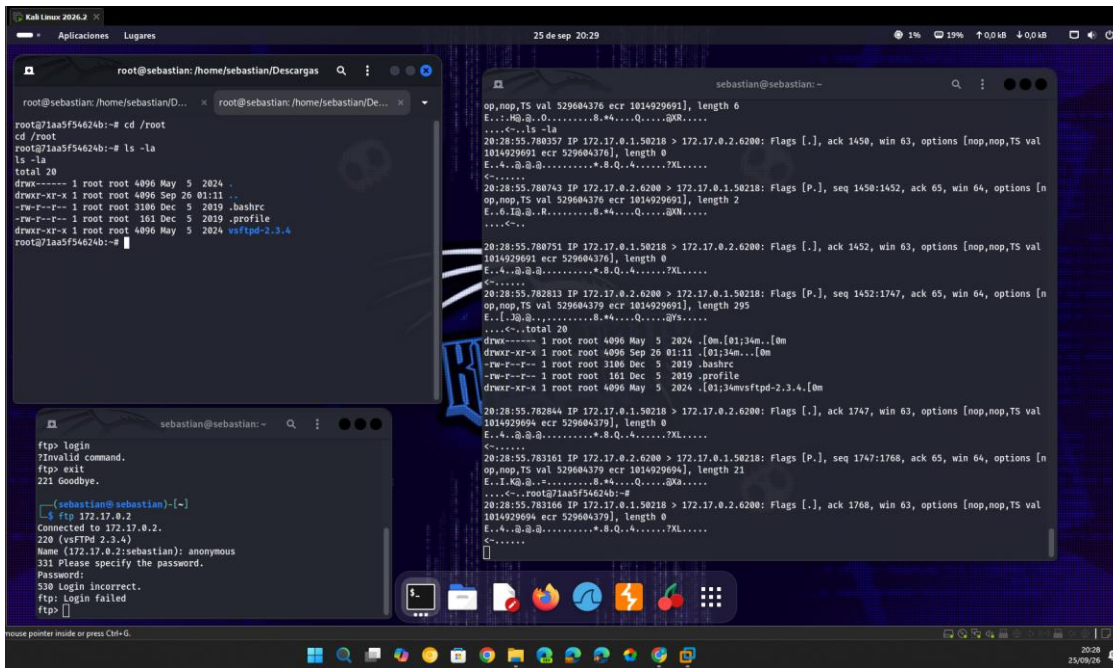


Figura 29

Figura 29. Comandos ejecutados (`cd /root`, `ls -la`) y su salida, capturados en claro.

Durante la exploración se buscó además un posible archivo de bandera (flag). El comando `cat flag.txt` devolvió `No such file or directory` en el directorio de trabajo, por lo que **no se localizó un archivo de bandera** en las rutas inspeccionadas; se continuó con una búsqueda de ficheros por patrón. La evidencia relevante de esta actividad no es una bandera, sino la **demonstración de la exposición de información en texto plano**, que es precisamente el riesgo que el ejercicio buscaba ilustrar.

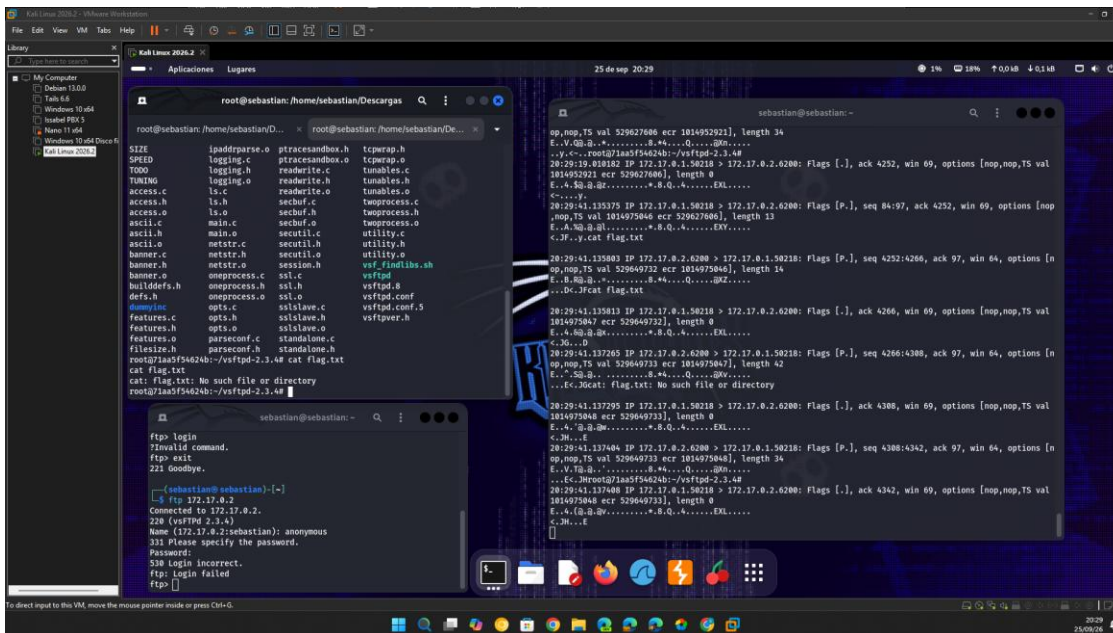


Figura 30

Figura 30. Búsqueda de un archivo de bandera: `cat flag.txt` → “No such file or directory”.

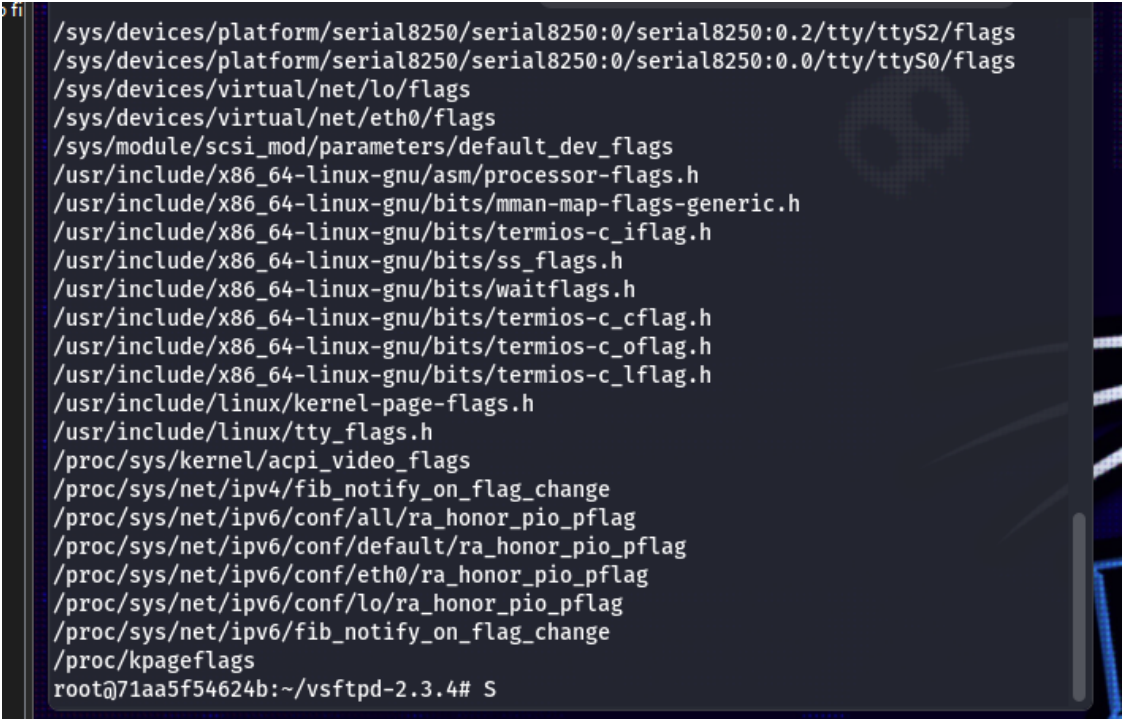


Figura 31

Figura 31. Enumeración de ficheros por patrón en `/proc`, `/sys` y `/usr/include`.

10. Documentación de comandos: propósito y evidencia

Conforme a la indicación de la actividad —investigar, ejecutar y documentar comandos comprendiendo la función de cada uno—, este apartado consolida los **comandos empleados durante el desarrollo del laboratorio** y, además, **veinte comandos adicionales** investigados y ejecutados de forma independiente. Cada comando se acompaña de su propósito y de la figura donde se evidencia su ejecución.

10.1 Comandos empleados en la práctica

N.º	Comando	Propósito / función	Evidencia
1	file	Identifica el tipo real de un archivo por su contenido, no por su extensión.	Fig. 8
2	unzip	Descomprime archivos en formato .zip.	Fig. 8
3	ls -la	Lista archivos con detalle: permisos,	Fig. 8

N.º	Comando	Propósito / función	Evidencia
		propietario, tamaño y ocultos.	
4	tree	Muestra la estructura de directorios en forma de árbol.	Fig. 9
5	chmod	Modifica los permisos de un archivo (otorgar ejecución al script).	Fig. 10
6	sudo	Ejecuta un comando con privilegios de superusuario.	Fig. 11
7	su	Cambia la sesión al usuario root.	Fig. 14
8	apt	Gestor de paquetes: actualiza el sistema e instala software.	Fig. 13
9	nmap	Escanea puertos, servicios y versiones de un objetivo.	Fig. 16
10	python3	Ejecuta scripts en Python (la prueba de concepto).	Fig. 19
11	hostname -I	Muestra las direcciones IP asignadas al equipo.	Fig. 20
12	whoami	Muestra el usuario efectivo de la sesión.	Fig. 25
13	script	Estabiliza la sesión de terminal (mejora la TTY).	Fig. 22
14	export	Define una variable de entorno (TERM).	Fig. 23
15	echo	Imprime texto o el valor de una variable.	Fig. 24
16	cd	Cambia el directorio de trabajo.	Fig. 29
17	cat	Muestra el contenido de un archivo.	Fig. 30
18	ftp	Cliente de línea de comandos para	Fig. 27

N.º	Comando	Propósito / función	Evidencia
		conectarse a un servidor FTP.	
19	tcpdump	Captura y analiza el tráfico de red de una interfaz.	Fig. 28
20	man	Muestra el manual de referencia de un comando.	Fig. 14
21	find	Busca archivos por nombre o patrón en el sistema de ficheros.	Fig. 31

10.2 Comandos adicionales investigados y ejecutados

Se seleccionaron veinte comandos **distintos de los anteriores**, relevantes para tres áreas del trabajo de un analista de seguridad: información del sistema, reconocimiento de red y análisis de archivos. Todos fueron ejecutados en Kali Linux; su evidencia se presenta en las figuras posteriores a la tabla.

N.º	Comando	Propósito / función	Evidencia
1	id	Muestra el UID, el GID y los grupos de la sesión actual.	Fig. 32
2	uname -a	Información del kernel y del sistema operativo (versión, arquitectura).	Fig. 32
3	uptime	Tiempo que lleva encendido el sistema y su carga promedio.	Fig. 32
4	free -h	Uso de memoria RAM y de intercambio (<i>swap</i>).	Fig. 32
5	df -h	Espacio total, usado y disponible de los sistemas de archivos.	Fig. 32
6	ps aux	Instantánea de todos los procesos en ejecución.	Fig. 32
7	top	Monitor de procesos y recursos en tiempo real.	Fig. 33

N.º	Comando	Propósito / función	Evidencia
8	ip a	Interfaces de red y direcciones IP asignadas.	Fig. 34
9	ip route	Tabla de enrutamiento (puerta de enlace y rutas).	Fig. 34
10	ss -tuln	Puertos y sockets en escucha (TCP/UDP).	Fig. 34
11	arp -a	Tabla ARP: correspondencia entre direcciones IP y MAC.	Fig. 35
12	ping	Comprueba la conectividad con un host mediante ICMP.	Fig. 35
13	tracert	Traza los saltos de red intermedios hasta un destino.	Fig. 35
14	nc (Netcat)	Abre conexiones, comprueba puertos y captura <i>banners</i> .	Fig. 38
15	wget	Descarga archivos desde la red por HTTP, HTTPS o FTP.	Fig. 36
16	dig	Realiza consultas al sistema de nombres de dominio (DNS).	Fig. 36
17	grep	Filtra las líneas que coinciden con un patrón.	Fig. 38
18	head / tail	Muestran las primeras o las últimas líneas de un archivo.	Fig. 37
19	wc	Cuenta líneas, palabras y bytes de un archivo.	Fig. 37
20	sha256sum / md5sum	Calculan el <i>hash</i> de un archivo para verificar su integridad.	Fig. 36

```
root@sebastian:~/home/sebastian
id
uid=0(root) gid=0(root) grupos=0(root)

root@sebastian:~/home/sebastian
uname -a
Linux sebastian 6.19.14-hali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.19.14-1-hali1 (2026-03-09) x86_64 GNU/Linux

root@sebastian:~/home/sebastian
uptime
21:13:35 up 12 min, 2 users, load average: 0,10, 0,16, 0,11

root@sebastian:~/home/sebastian
free -h
             total        usada      libre compartido  búf/cache disponible
Mem:          7,7Gi          1,4Gi          6,3Gi          17Mi          1,9Gi          6,3Gi
Inter:         5,1Gi           0B          5,1Gi

root@sebastian:~/home/sebastian
df -h
Filesystem              Tamaño Usados  Disp Usos% Montado en
udev                    3,9G          0  3,9G   0% /dev
tmpfs                   791M          0  791M   0% /run
/dev/sda1                93G          0  93G   0% /
tmpfs                   3,9G          0  3,9G   0% /dev/shm
tmpfs                    1,0M          0  1,0M   0% /run/credentials/systemd-journald.service
tmpfs                   3,9G          0  3,9G   0% /tmp
tmpfs                   791M          0  791M   0% /run/user/1000
tmpfs                   791M          0  791M   0% /run/user/0

root@sebastian:~/home/sebastian
ps aux
USER          PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.3  0.2 26320 36224 ?        Ss   21:01   0:02 /sbin/init splash
root           3  0.0  0.0  0  0 0 0      S   21:03   0:00 [kthreadd]
root          33  0.0  0.0  0  0 0 0      S   21:03   0:00 [pool_workqueue_release]
root          41  0.0  0.0  0  0 0 0      S   21:03   0:00 [kworker/R-cpu_up]
root          51  0.0  0.0  0  0 0 0      S   21:03   0:00 [kworker/R-sync_wq]
root          61  0.0  0.0  0  0 0 0      S   21:03   0:00 [kworker/R-kvfree_rcu_reclaim]
root          71  0.0  0.0  0  0 0 0      S   21:03   0:00 [kworker/R-slab_flushq]
root          81  0.0  0.0  0  0 0 0      S   21:03   0:00 [kworker/R-netns]
root          10  0.0  0.0  0  0 0 0      S   21:03   0:00 [kworker/R-0M-events_highpri]
root          11  0.0  0.0  0  0 0 0      S   21:03   0:00 [kworker/R-0M-events_highpri]
root          12  0.0  0.0  0  0 0 0      S   21:03   0:00 [kworker/u512:0-ipvs_addrconf]
root          13  0.0  0.0  0  0 0 0      S   21:03   0:00 [kworker/R-mm_percpu_wq]
root          14  0.0  0.0  0  0 0 0      S   21:03   0:00 [ksoftirqd/0]
root          15  0.0  0.0  0  0 0 0      S   21:03   0:00 [rcu_preempt]
```

Figura 32

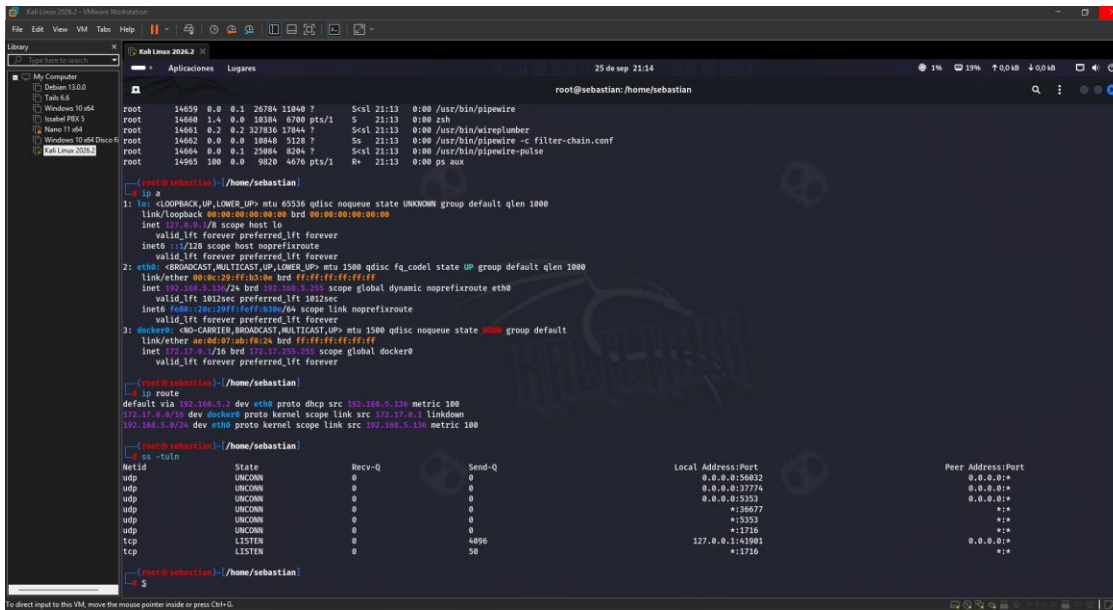
Figura 32. Información del sistema: id, uname -a, uptime, free -h, df -h y ps aux.

```
top - 21:08:00 up 6 min, 2 users, load average: 0,27, 0,27, 0,13
Tasks: 381 total, 1 ejecutar, 380 hibernar, 0 detenido, 0 zombie
%Cpu(s):  0,0 us,  0,0 sy,  0,0 ni, 99,4 id,  0,0 wa,  0,0 hi,  0,2 si,  0,0 st
MiB Mem : 7987,5 total, 4857,5 libre, 1440,2 usado, 1890,4 búf/cache
MiB Intercambio: 5238,0 total, 5238,0 libre,  0,0 usado, 6407,3 dispon Mem

PID  USUARIO  PR  NI  VIRT  RES  SHR  S  %CPU  %MEM    TIME+  COMMAND
2929 sebasti+ 20  0  735376 511220 170932 S   0,6  0,3  0:45.86 gnome-shell
15 root      20  0  0  0  0  0  0  0,3  0,0  0:00:10 rcu_preempt
1267 root      20  0  0  0  0  0  0  0,1  0,3  0,0  0:00:02 kworker/1:3-events
2092 sebasti+ 20  0 1015272 120536 93572 S   0,3  1,5  0:01:09 vmtoutid
11845 root    20  0  0  0  0  0  0  0,3  0,0  0:00:01 kworker/5:8-events
11620 sebasti+ 20  0 1515616 69952 54576 S   0,3  0,9  0:00:06 gnome-terminal-
1 root      20  0 26320 36224 11780 S   0,0  0,2  0:02:44 systemd
2 root      20  0  0  0  0  0  0  0,5  0,0  0:00:00 kthreadd
3 root      20  0  0  0  0  0  0  0,5  0,0  0:00:00 pool_workqueue_release
4 root      20  0  0  0  0  0  0  0,1  0,0  0:00:00 kworker/R-cpu_gp
5 root      20  0  0  0  0  0  0  0,1  0,0  0:00:00 kworker/R-sync_wq
6 root      20  0  0  0  0  0  0  0,1  0,0  0:00:00 kworker/R-kvfree_rcu_reclaim
7 root      20  0  0  0  0  0  0  0,1  0,0  0:00:00 kworker/R-slab_flushq
8 root      20  0  0  0  0  0  0  0,1  0,0  0:00:00 kworker/R-netns
10 root     20  0  0  0  0  0  0  0,1  0,0  0:00:00 kworker/0:0-events_highpri
11 root     20  0  0  0  0  0  0  0,1  0,0  0:00:00 kworker/0:1-events
12 root     20  0  0  0  0  0  0  0,1  0,0  0:00:00 kworker/u512:0-ipvs_addrconf
13 root     20  0  0  0  0  0  0  0,1  0,0  0:00:00 kworker/R-mm_percpu_wq
14 root     20  0  0  0  0  0  0  0,0  0,0  0:00:00 ksoftirqd/0
16 root     20  0  0  0  0  0  0  0,5  0,0  0:00:04 rcu_exp_gp_kthread_worker/1
17 root     20  0  0  0  0  0  0  0,5  0,0  0:00:02 rcu_exp_gp_kthread_worker
18 root     20  0  0  0  0  0  0  0,0  0,0  0:00:00 migration/0
19 root     20  0  0  0  0  0  0  0,5  0,0  0:00:00 idle_inject/0
20 root     20  0  0  0  0  0  0  0,5  0,0  0:00:00 cpmu/0
21 root     20  0  0  0  0  0  0  0,0  0,0  0:00:00 cpmu/1
22 root     20  0  0  0  0  0  0  0,5  0,0  0:00:00 idle_inject/1
23 root     20  0  0  0  0  0  0  0,5  0,0  0:00:00 migration/1
24 root     20  0  0  0  0  0  0  0,0  0,0  0:00:00 ksoftirqd/1
25 root     20  0  0  0  0  0  0  0,1  0,0  0:00:03 kworker/1:0-events
26 root     20  0  0  0  0  0  0  0,1  0,0  0:00:00 kworker/1:0-events_highpri
27 root     20  0  0  0  0  0  0  0,5  0,0  0:00:00 cpmu/2
28 root     20  0  0  0  0  0  0  0,5  0,0  0:00:00 idle_inject/2
29 root     20  0  0  0  0  0  0  0,5  0,0  0:00:00 migration/2
30 root     20  0  0  0  0  0  0  0,0  0,0  0:00:01 ksoftirqd/2
31 root     20  0  0  0  0  0  0  0,1  0,0  0:00:05 kworker/2:0-cgroup_release
32 root     20  0  0  0  0  0  0  0,1  0,0  0:00:00 kworker/2:0-events_highpri
33 root     20  0  0  0  0  0  0  0,5  0,0  0:00:00 cpmu/3
34 root     20  0  0  0  0  0  0  0,5  0,0  0:00:00 idle_inject/3
35 root     20  0  0  0  0  0  0  0,5  0,0  0:00:00 migration/3
```

Figura 33

Figura 33. Monitor de procesos y recursos en tiempo real con top.



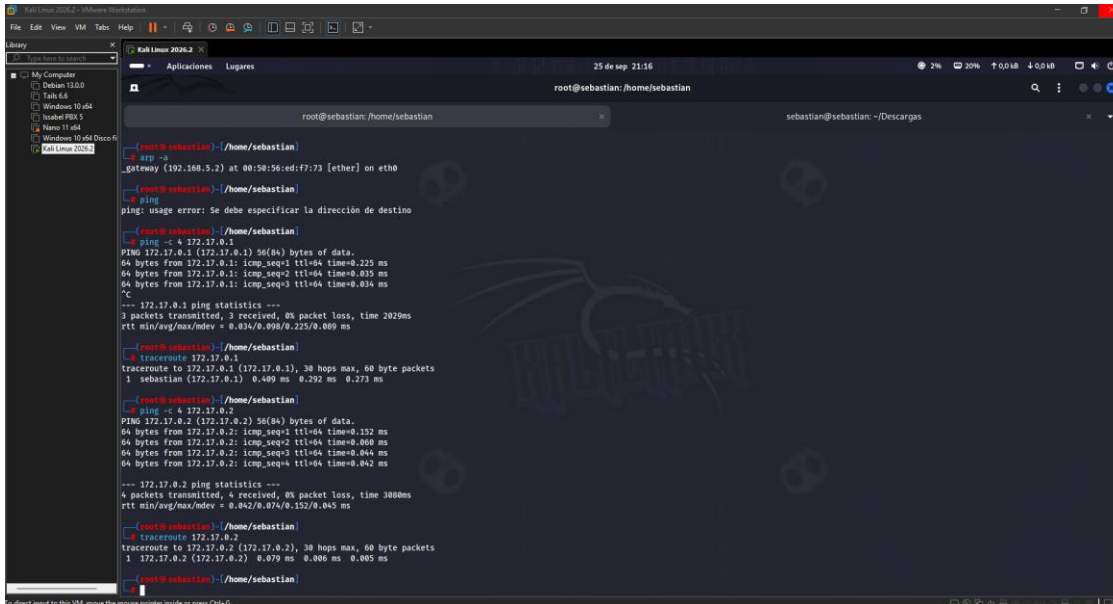
```
root@sebastian:/home/sebastian

1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:1f:3b:06 brd ff:ff:ff:ff:ff:ff
    inet 192.168.5.130/24 brd 192.168.5.255 scope global dynamic noprefixroute eth0
        valid_lft 1812sec preferred_lft 1812sec
    inet6 fe80::c29:1f3b:06:00/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: docker0: <CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether aa:08:07:aa:f8:26 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever

root@sebastian:/home/sebastian
# ss -tuln
State          Recv-Q         Send-Q         Local Address:Port      Peer Address:Port
udp            UNCONN          0              0.0.0.0:56832            0.0.0.0:*
udp            UNCONN          0              0.0.0.0:37774            0.0.0.0:*
udp            UNCONN          0              0.0.0.0:51553            0.0.0.0:*
udp            UNCONN          0              *:36677                  *:*
udp            UNCONN          0              *:5253                    *:*
udp            UNCONN          0              *:1716                    *:*
tcp            LISTEN          4096           127.0.0.1:41901          0.0.0.0:*
tcp            LISTEN          0              *:1716                    *:1716
```

Figura 34

Figura 34. Configuración de red: ip a, ip route y ss -tuln.



```
root@sebastian:/home/sebastian
# arp -a
Gateway (192.168.5.2) at 00:58:56:ed:f7:73 [ether] on eth0

root@sebastian:/home/sebastian
# ping
ping: usage error: Se debe especificar la dirección de destino

root@sebastian:/home/sebastian
# ping -c 4 172.17.0.1
PING 172.17.0.1 (172.17.0.1) 56(84) bytes of data.
64 bytes from 172.17.0.1: icmp_seq=1 ttl=64 time=0.225 ms
64 bytes from 172.17.0.1: icmp_seq=2 ttl=64 time=0.835 ms
64 bytes from 172.17.0.1: icmp_seq=3 ttl=64 time=0.834 ms
^C
--- 172.17.0.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2029ms
rtt min/avg/max/ndev = 0.834/0.898/0.225/0.889 ms

root@sebastian:/home/sebastian
# traceroute 172.17.0.1
traceroute to 172.17.0.1 (172.17.0.1), 30 hops max, 60 byte packets
1 sebastian (172.17.0.1) 0.409 ms 0.292 ms 0.273 ms

root@sebastian:/home/sebastian
# ping -c 4 172.17.0.2
PING 172.17.0.2 (172.17.0.2) 56(84) bytes of data.
64 bytes from 172.17.0.2: icmp_seq=1 ttl=64 time=0.152 ms
64 bytes from 172.17.0.2: icmp_seq=2 ttl=64 time=0.860 ms
64 bytes from 172.17.0.2: icmp_seq=3 ttl=64 time=0.844 ms
64 bytes from 172.17.0.2: icmp_seq=4 ttl=64 time=0.842 ms
--- 172.17.0.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3880ms
rtt min/avg/max/ndev = 0.842/0.874/0.152/0.845 ms

root@sebastian:/home/sebastian
# traceroute 172.17.0.2
traceroute to 172.17.0.2 (172.17.0.2), 30 hops max, 60 byte packets
1 172.17.0.2 (172.17.0.2) 0.079 ms 0.806 ms 0.805 ms
```

Figura 35

Figura 35. Diagnóstico de red: arp -a, ping y traceroute (a 172.17.0.1 y al objetivo 172.17.0.2).

Hallazgo	Vulnerabilidad	Riesgo	Impacto	Control / mitigación
		autenticación		3.x); retirar binarios comprometidos
Servicio FTP expuesto	Protocolo sin cifrado (texto plano)	Intercepción de credenciales y datos (sniffing)	Pérdida de confidencialidad	Migrar a SFTP/FTPS ; segmentar y cifrar el canal
Superficie de red amplia	Servicio innecesario accesible	Mayor exposición a exploits públicos	Aumento de la probabilidad de ataque	Hardening : cerrar puertos no usados, firewall, mínima exposición
Versión de software revelada	Fuga de información de versión (banner)	Facilita la búsqueda de exploits dirigidos	Reconocimiento acelerado del atacante	Ocultar o normalizar banners; gestión de parches
Falta de monitorización	Ausencia de detección	Explotación no advertida	Persistencia del atacante	IDS/IPS , registro y alertas de conexiones anómalas (p. ej. a 6200/tcp)

13. Capacidades de virtualización

- **Ventajas de ejecutar Kali como máquina virtual.** Aislamiento respecto al equipo anfitrión, instantáneas (*snapshots*) para revertir cambios, portabilidad, y la posibilidad de destruir y recrear el entorno sin afectar hardware real. Este laboratorio lo evidenció de primera mano: ante la ruptura de Kali 2025.2 se pudo **recuperar** el entorno reinstalando la versión 2026.2, sin ninguna consecuencia para el sistema anfitrión.
- **Aporte de una red aislada.** Confina el tráfico ofensivo al laboratorio y evita que la máquina vulnerable o las pruebas alcancen redes institucionales o de terceros. Si el objetivo estuviera en modo **punto a punto (bridge)** contra la red institucional, un servicio vulnerable como vsftpd 2.3.4 quedaría expuesto a toda la red corporativa, con riesgo real de compromiso y de uso como punto de pivote.
- **Utilidad del snapshot.** Permite volver a un estado limpio y conocido antes de una prueba y restaurarlo si esta altera el entorno, garantizando así la repetibilidad del escenario.
- **Máquina virtual frente a contenedor.** Una máquina virtual virtualiza hardware completo y ejecuta su propio kernel (aislamiento fuerte); un contenedor —como el objetivo de esta práctica— comparte el kernel del anfitrión y aísla a nivel de procesos y espacios de nombres (más ligero, pero con aislamiento menor). Por eso el objetivo se ejecuta como contenedor Docker (71aa5f54624b) mientras que el atacante es una máquina virtual completa.

14. Caso empresarial

Una empresa detecta que uno de sus servidores mantiene un servicio FTP antiguo y decide reproducir la situación en un laboratorio virtual antes de intervenir el sistema productivo.

- **Ventajas de reproducir el escenario en una máquina virtual.** Permite validar la vulnerabilidad y el efecto de los controles **sin riesgo** para producción, de forma repetible y recuperable.

- **Información previa a recopilar.** Inventario del activo, versión exacta del servicio, configuración, exposición de red, autorizaciones y ventana de mantenimiento, y respaldos verificados.
- **Controles en el servidor productivo.** Actualización o parcheo del servicio, migración a SFTP/FTPS, segmentación de red y firewall, principio de mínima exposición, monitorización y un proceso continuo de gestión de vulnerabilidades.
- **Evidencias a conservar durante la auditoría.** Resultados de escaneos, capturas de tráfico, registros de comandos y salidas, cronología y autorizaciones (cadena de custodia).
- **Procedimiento de recuperación si la prueba altera el laboratorio.** Restaurar desde un *snapshot* o volver a desplegar el contenedor, devolviendo el entorno a su estado inicial conocido.

15. Conclusiones técnicas

1. Se completó de principio a fin el ciclo de una evaluación de seguridad controlada —preparación, reconocimiento, enumeración, investigación, validación y análisis— sobre un objetivo autorizado y aislado (172.17.0.2).
2. El servicio **vsftpd 2.3.4** (puerto 21/tcp) resultó vulnerable a **CVE-2011-2523**; la prueba de concepto controlada demostró la ejecución de comandos con privilegios de **root**, evidenciando la diferencia entre *identificar* y *demostrar* una vulnerabilidad.
3. El análisis de tráfico confirmó que FTP transmite en **texto plano**, un riesgo de confidencialidad independiente de la puerta trasera; su mitigación pasa por cifrar el canal (SFTP/FTPS) y por una gestión de parches oportuna.
4. Las incidencias técnicas (la ruptura de Kali 2025.2 y los errores de permisos y de sintaxis) reforzaron el valor de la **virtualización** —recuperación y repetibilidad— y de una metodología sistemática de resolución de problemas.
5. Kali Linux se confirma no como un simple conjunto de herramientas ofensivas, sino como una **plataforma de aprendizaje, evaluación y validación controlada** de la seguridad.

16. Referencias

- Exploit Database. (s. f.). *vsftpd 2.3.4 — Backdoor Command Execution (EDB-ID 49757)*. <https://www.exploit-db.com/exploits/49757>
- MITRE. (s. f.). *CVE-2011-2523*. Common Vulnerabilities and Exposures. <https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2011-2523>
- National Institute of Standards and Technology. (s. f.). *CVE-2011-2523 Detail — National Vulnerability Database (NVD)*. <https://nvd.nist.gov/vuln/detail/CVE-2011-2523>
- Nmap Project. (s. f.). *Nmap Reference Guide*. <https://nmap.org/book/man.html>
- The Tcpdump Group. (s. f.). *tcpdump(1) man page*. <https://www.tcpdump.org/manpages/tcpdump.1.html>
- The vsftpd Project. (s. f.). *vsftpd — Very Secure FTP Daemon*. <https://security.appspot.com/vsftpd.html>